Lab 3 - Instructions

Piecewise functions
Software 1 - Mathematics
Information and Communication Technology
Metropolia University of Applied Sciences

What you'll learn

In this laboratory exercise, you will master:

- **Boolean array operations** in NumPy - the foundation for creating conditional logic with arrays
- **Piecewise functions** - mathematical functions defined by different expressions in different intervals
- **Array multiplication with boolean masks** - using True/False arrays to selectively apply mathematical operations
- **Visual verification** of mathematical functions through proper plotting techniques

These skills are essential for signal processing, data filtering, and modeling real-world phenomena that have different behaviors in different conditions.

Expected solutions

By the end of this lab, you should be able to:

1. **Problem 1**: Create a piecewise continuous function (triangle-shaped) using boolean array operations that correctly handle 4 different regions with appropriate mathematical expressions

2. **Problem 2**: Build another piecewise function using the same techniques, producing a smooth, continuous graph without jumps or discontinuities

Both solutions should:

- Use **only boolean array operations** (no `if-statements` or `np.vectorize`)
- Produce **smooth graphs** with sufficient data points (step size ≤ 0.01)
- Display **proper decorations** (title, axis labels, legend, grid)
- Match the **exact mathematical specifications** for your personalized problem

Before you begin

Critical Setup Requirements

1. File Organization

- Ensure `labx.py` is downloaded and placed in the **same folder** as your notebook
- Your notebook must be saved in this project folder
- Test the import: `import labx as mx` should run without errors

2. Get Your Personalized Problems

Each student receives unique problems based on their student ID. You **must**:

- **Set your `student_id`** in the Setup section
- **Run the code cells** that display your specific problems:
  - `txt = mx.lab_3_problem_1(student_id)`
  - `display(Markdown(txt))`
- **Read carefully** the mathematical functions displayed for YOUR student ID

3. Complete the Moodle Quiz

Before attempting these problems, complete Moodle Quiz Lab 3 to practice the fundamental boolean operations.

Setup notes

Required Libraries

```
from IPython.display import Markdown, display
import numpy as np
import matplotlib.pyplot as plt
import labx as mx
```

Important: Import these libraries only **once** at the beginning of your notebook. Do not repeat import statements in later cells.

Creating Proper x-arrays

Common mistake

```
x = np.arange(-10, 10) # Only 20 points - creates jagged graphs!
```

Correct approach

```
x = np.arange(-10, 10, 0.01) # 2000 points - creates smooth graphs
# OR
x = np.linspace(-10, 10, 2000) # Explicitly specify number of points
```

Why this matters

Piecewise functions need many points to display smooth transitions. With too few points, your graphs will look rough and discontinuous.

Problem 1

Understanding the Task

You will create a **piecewise function** using boolean array operations. Think of this like a Swiss Army knife - the function behaves differently in different regions of x, but it's all defined in one expression.

Step-by-Step Approach

Step 1: Display YOUR specific problem

```
txt = mx.lab_3_problem_1(student_id)
display(Markdown(txt))
```

Read the mathematical definition carefully. It typically defines 4 regions with different behaviors.

Step 2: Create the x-array with sufficient points

```
x = np.linspace(-10, 10, 2000) # Dense array for smooth plotting
```

Step 3: Understand boolean operations

Boolean operations create True/False masks:

```
mask = (x >= 0.6) & (x < 2.6) <# True where condition is met
```

When multiplied with numbers, boolean values behave as: - `True` → 1 (includes the value) - `False` → 0 (excludes the value)

Step 4: Build the piecewise function

The key insight: combine multiple conditional expressions using addition.

Example pattern (not the actual problem):

```
# For a function defined as:
# f(x) = 0,            when x < 0
# f(x) = x,            when 0 ≤ x < 2
# f(x) = 2,            when x ≥ 2

y = (x >= 0) * (x < 2) * x + (x >= 2) * 2
```

Breaking this down:

- `(x >= 0) * (x < 2) * x` → applies x only in the interval [0, 2)
- `(x >= 2) * 2` → applies value 2 when x ≥ 2
- Addition combines the pieces

Step 5: Verify visually

The solution graph should look like a **triangle**. If your graph doesn't match this shape:

- Check your boolean conditions (≥ vs >, boundaries)
- Verify your mathematical expressions match the problem
- Draw the function by hand first to understand the expected shape

Step 6: Create a proper plot

```
plt.figure(figsize=(8, 5))
plt.plot(x, y, label='f(x)')
plt.xlabel('x')
plt.ylabel('f(x)')
plt.title('Problem 1: Piecewise Function')
plt.legend()
plt.grid(True)
plt.show()
```

Problem 2

Understanding the Task

Like Problem 1, but with different mathematical expressions and boundary conditions.

Step-by-Step Approach

Step 1: Display YOUR specific problem

```
txt = mx.lab_3_problem_2(student_id)
display(Markdown(txt))
```

Step 2: Use the same x-array

You can reuse the x-array from Problem 1 if it covers the required range. Otherwise, create a new one with adequate density, for example:

```
x = np.linspace(-10, 10, 2000)
```

Step 3: Apply the same technique

Follow the same pattern as Problem 1:

1. Identify all the regions and their mathematical expressions
2. Create boolean masks for each region
3. Multiply each expression by its corresponding mask
4. Add all pieces together

Step 4: Debug discontinuities

If your graph has **sudden jumps or gaps**:

- Check that adjacent regions share boundary values correctly
- Verify that conditions use ≤ vs < appropriately
- Ensure all regions are covered (no “gaps” in the domain)
- Test boundary points manually: what should happen at x = boundary?

Example debugging

```
# If there's a jump at x = 2, test:
x_test = np.array([1.99, 2.0, 2.01])
y_test = # your function
print(x_test, y_test)  # Should show continuous values
```

Common errors and solutions

Error 1: ModuleNotFoundError: No module named ‘labx’

Problem

The notebook cannot find the `labx.py` file.

Solutions

- Verify `labx.py` is in the **same folder** as your notebook
- Check that you saved your notebook in the correct project folder
- In PyCharm, check your project structure in the Project panel
- Try restarting the kernel and running all cells again

Error 2: Graph doesn’t look like the expected shape

Problem

Your piecewise function doesn’t match the description (e.g., not triangular).

Solutions

- **Sketch it by hand first**
  - Draw each piece of the function separately, then combined
- **Verify each region separately**

```
# Test each piece individually
piece1 = (x < 0.6) * 0
piece2 = (x >= 0.6) * (x < 2.6) * (-0.6 + x)
plt.plot(x, piece1, label='Piece 1')
plt.plot(x, piece2, label='Piece 2')
plt.legend()
plt.show()
```

- **Check boundary values**
  - Does your function produce the correct value at each boundary point?
  - Review boolean operations: Are you using & (and) and | (or) correctly?

Error 3: Graph is rough/jagged

Problem

Your plot looks like a connect-the-dots drawing.

Solution

Increase the number of points in your x-array:

```
# Too few points
x = np.arange(-10, 10)  # Only 20 points

# Sufficient points
x = np.arange(-10, 10, 0.01)  # 2000 points
# OR
x = np.linspace(-10, 10, 2000)
```

Error 4: Used if-statements or np.vectorize

Problem

Your solution uses `if`, `elif`, `else` statements or `np.vectorize()`.

Solution

This exercise specifically requires **boolean array operations**. Rewrite using the pattern:

```
# Wrong approach
def piecewise(x):
    if x < 0:
        return 0
    elif x < 2:
        return x
    else:
        return 2

# Correct approach
y = (x >= 0) * (x < 2) * x + (x >= 2) * 2
```

Why

Boolean operations work on entire arrays at once (vectorized), making them much faster and more Pythonic than loops or conditional statements.

Error 5: Your answers don't correspond to the displayed equations

Problem

You're solving the wrong problem.

Solutions

- Make sure you ran the code to display YOUR specific problems:

```
txt = mx.lab_3_problem_1(student_id)

display(Markdown(txt))
```

- Don't solve the example from the Instructions notebook
- Verify your `student_id` is set correctly
- Read the output of the display command - those are YOUR equations

Error 6: Sudden jumps or discontinuities in the graph

Problem

The graph shows unexpected vertical jumps.

Solutions

- *Check overlapping conditions*. Make sure regions don't overlap

```
# Problem: regions overlap
piece1 = (x >= 0) * (x <= 2) * expression1
piece2 = (x >= 2) * (x <= 4) * expression2
# x=2 counted twice!

# Solution: use < and ≥ appropriately
piece1 = (x >= 0) * (x < 2) * expression1
piece2 = (x >= 2) * (x <= 4) * expression2
```

- *Verify boundary values match*. At boundaries, the function value from the left should equal the value from the right
- *Test at boundaries*. Manually check what your function returns at critical x-values

Error 7: Code is too complicated

Problem

Your solution uses loops, custom functions, or complex logic.

Solution

Boolean array operations make this simple. The entire piecewise function should be **one line** (or split across a few lines for readability):

```
y = ((condition1) * expression1 +
     (condition2) * expression2 +
     (condition3) * expression3 +
     (condition4) * expression4)
```

If your code is much longer, review the lecture materials on boolean operations.

Additional resources

NumPy Documentation

- Boolean array indexing
- numpy.linspace
- numpy.arange

External tutorials

- Boolean indexing in NumPy -
- Boolean NumPy arrays

Matplotlib

- Pyplot tutorial
- matplotlib.pyplot

Final checklist before submission

- ☐ `labx.py` file is in the same folder as your notebook
- ☐ You've set your correct `student_id`
- ☐ You've run the code to display YOUR personalized problems
- ☐ Your graphs are smooth (used sufficient data points)
- ☐ Problem 1 graph looks like a triangle
- ☐ Problem 2 graph has no sudden jumps or discontinuities
- ☐ You used **only boolean array operations** (no if-statements)
- ☐ All graphs have titles, axis labels, legends, and gridlines
- ☐ You tested your code by running "Run All" from the beginning
- ☐ Your notebook is saved and ready to submit

Good luck! Remember: boolean operations are powerful tools that make working with arrays elegant and efficient.